

Course Code	Course Title	L	T	P	C
BACSE101	Problem Solving Using Python	0	0	4	2
Pre-requisite		Syllabus Version			
NIL		1.0			
Course Objectives					
1. To provide the basics and fundamentals of the python programming language and introduce algorithmic skills through programming constructs 2. To impart python collections for efficient data handling and processing and inculcate modular and recursive programming approaches for developing software applications 3. To teach python libraries for data analysis and manipulations					
Course Outcomes					
At the end of the course, students should be able to 1. Identify appropriate algorithmic approach, data representation, control constructs in developing software solutions for solving real-world problems. 2. Inculcate data handling techniques using python collections and modular programming strategies for developing multi-disciplinary software applications 3. Idealize the importance of python libraries for handling, manipulating and analyzing multi-faceted real world problem domains.					
Module:1	Introduction to Problem Solving and Python Fundamentals				6 hours
Fundamentals of Problem Solving: Problem Analysis Chart - Robust and Boundary Test Cases – Algorithm- Flowchart - Pseudocode Data Types: Numeric, Boolean and Strings - Regular Expression - Operators - Expressions - Built in Functions					
Module:2	Problem Solving Approaches and Constructs for Controlling Program Flow				14 hours
Problem Solving Approaches: Top-down - Bottom-up - Divide & Conquer - Backtracking Conditional Statements -Branching-Looping - Break, Continue and Pass Statements					
Module:3	Data Organization and Manipulation using Collections				14 hours
Data Handling Strategies Using Lists, Tuples, Dictionary and Set- Data Comprehension - Iterators - Data Selection - Ordered, Unordered and Unique Data Organization - Data Modification - Grouping and Categorization – Searching Techniques: Linear Search, Binary Search - Sorting Techniques: Bubble Sort, Selection Sort- Insertion Sort- Quick Sort- Merge Sort.					
Module:4	Modular Programming				14 hours
Fundaments of Function: Definition, Call, Return Value – Parameters and Arguments: Positional arguments, Keyword arguments, Parameters with Default Values, Arbitrary Arguments – Local and Global Scope - Lambda Functions – Decorators - Recursive Functions- Menu driven implementations: Stacks- Queues.					
Module:5	Data Processing using Numpy and Pandas				10 hours
Numpy: Array Operations – Mathematical Functions - Pandas: Handling Files - Creating Dataframes – Data Cleaning – Filtering – Selection – Grouping – Sorting – Aggregation – Merging.					
Module:6	Contemporary Issues				2 hours

		Total Lecture hours:	60 hours
Text Books			
1	John V. Guttag (2021). Introduction to computation and programming using Python (3rd ed.) The MIT Press		
2	Eric Matthes (2022). Python Crash Course (3rd ed.). A Hands-On, Project-Based Introduction to Programming.		
Reference Books			
1.	Luciano Ramalho (2022). Fluent Python (2nd ed.). O'Reilly Media, Inc		
2.	Charles R. Severance (2024). Python for Everybody (1 st ed.). Shroff Publishers.		
Mode of Evaluation: QUIZ/TEAM PROJECT/PAT/CAT/FAT			
List of Experiments (Indicative)			
Fundamentals of Problem Solving: Problem Analysis Chart - Robust and Boundary Test Cases – Algorithm- Flowchart - Pseudocode Data Types: Numeric, Boolean and Strings - Regular Expression - Operators - Expressions - Built in Functions (6 hours)			
1.	Subsidy Computation Agricultural subsidies are offered by the Indian government to farmers for improving crop yields and benefiting them. To facilitate the farmers in finding the subsidy amount, develop a Python code to calculate the subsidy based on the crop the farmer grows . The farmers can grow one of these three types of crops: Rice, Wheat, or Maize. Each crop has a different formula for calculating the subsidy: Formula: Rice: Subsidy= (Yield × Market Rate) + 100 Wheat: Subsidy= (Yield × Market Rate) + 200 Maize: Subsidy= (Yield × Market Rate) + 300 Use menu driven approach to implement the same.		
2.	Climate Change Data Analyzer Scientists track daily temperatures as a string of values separated by spaces (e.g., "30.5 32.1 29.8"). Write a Python program that extracts the temperatures, converts them into float values, and finds the highest, lowest, and average temperature. Rules & Constraints: i. Input is a string of space-separated float values (e.g., "30.5 31.2"). ii. All values must be valid floating-point numbers. iii. The list must have at least two values. iv. Temperatures must be within a realistic range: -100.0°C to 60.0°C. v. The program should round the average to two decimal places. vi. If the input is invalid (non-numeric, empty, out-of-range), return an error message.		
3	Gaming Leaderboard Username Validator In an online game, players choose usernames for the leaderboard. To make sure all usernames are clean and fair, they must follow these rules: i. The username must start with a letter (A–Z or a–z). ii. It can contain letters, numbers, and underscores (_) only. iii. It cannot start with a number. iv. No other symbols or spaces are allowed. v. The length must be between 5 and 15 characters.		

	Write a Python program using regular expressions to check if a username is valid or invalid. If invalid, display a message "Invalid Name"
4.	Number Game In Numeria, Alchemists must build an arithmetic expression using given numbers and operations to match a target. Use each number once, with +, -, *, or /. If no exact match, return the expression closest to the target. If multiple expressions give the same best result, return "No single solution" Rules: - Use all numbers once. - Only +, -, *, / allowed. - Division must result in integers only. - Left-to-right evaluation only (no brackets) - All numbers must be positive integers. - Return one valid expression, or "No single solution" if ties exist.
Problem Solving Approaches: Top-down - Bottom-up - Divide & Conquer - Backtracking Conditional Statements – Branching - Looping - Break, Continue and Pass Statements (14 hours)	
5.	Drone Delivery A delivery drone needs to find the shortest path from a warehouse (starting point) to a delivery location (destination) in a 2D grid ($N \times N$) city map, where 0 represents open paths and 1 represents obstacles. The drone can move up, down, left, or right, but it cannot pass through restricted zones (obstacles). Implement an algorithm (only) using a top-down approach to find the shortest path from the warehouse to the delivery location. - If the destination is reached, return the path length. - If the current cell is an obstacle or out of bounds, return an invalid path value. - Try moving in all four directions (up, down, left, right). - Use memoization to store results of already visited paths.
6.	Task Completion Time A group of students is participating in a coding competition, where they need to solve a set of problems as quickly as possible. Each student has a recorded completion time (in minutes) for each problem. The goal is to identify the student who solved their problems in the shortest total time. Implement an algorithm using Divide and Conquer problem solving approach to determine the student with the lowest total completion time. Input: A list of students with their total completion times. Base Case: If there is only one student, return them as the fastest. Divide: Split the student list into two halves. Conquer: Recursively find the fastest student in both halves. Combine: Compare the two selected students and return the one with the lower total time.
7.	Rental Computation Maya is developing a pricing system for a photography studio that rents out different types of cameras and needs to calculate the rental cost based on the equipment type and duration. The system should apply discounts based on the rental cost thresholds: Video Camera: The rental rate is ₹250 per hour. If the total rental cost exceeds ₹1500, a 12% discount is applied.

	ii. Drone Camera: The rental rate is ₹300 per hour. If the total rental cost exceeds ₹2000, a 15% discount is applied. Write a Python code to compute the rentalCost for VideoCamera, and DroneCamera and apply the respective discount based on the rental duration and the type of camera.
8.	Leader Selection (M2_CO1) In the city of Solaria, a group of n scholars sit in a circle. Each scholar has a number from 1 to n. The selection for the next Guardian of Harmony works like this: - Scholar 1 tells Scholar 2 to leave the circle. - Then the next scholar (still in the circle) tells the person next to them to leave. - This continues in a circle — each person removes the one next to them — until only one scholar is left. Determine which scholar will be the last one standing
9.	A Analyze health risk Analyze health risks based on multiple individuals' BMI (Body Mass Index), blood pressure, and heart rate. Using a loop to collect data for multiple individuals. Calculate BMI using $BMI = \text{weight} / (\text{height} ** 2)$, use conditional statements to classify BMI as low, medium or high. Low BMI (underweight): Less than 18.5 kg/m² Medium BMI (normal weight): 18.5 - 24.9 kg/m² High BMI (overweight): 25 kg/m² and above A "low" blood pressure is generally considered to be below 90/60 mmHg, while a "medium" or normal blood pressure is between 90/60 mmHg and 120/80 mmHg, otherwise high. For heart rate, a "low" resting rate is under 60 beats per minute (bpm), and a "medium" or normal resting rate is between 60-100 bpm, otherwise high. Additionally, check if blood pressure (systolic/diastolic) and heart rate fall within healthy ranges, categorizing individuals into Low, Moderate, or High health risk levels. Write a python code to display the BMI, blood pressure and heart rate. If two or more health risks are high, then the risk level is high. If any one of the health risks is high, then the risk level is medium. Otherwise, it is under low risk. Finally, display a summary report showing the number of individuals in each risk category.
10.	P Process DNA sequence DNA sequences are primarily analyzed by scientists, researchers, and medical professionals in fields like genetics, molecular biology, and medicine, to understand genetic variations, diagnose diseases. Write a Python program to process and analyze the DNA sequences by performing various operations based on nucleotide sequence rules. The program should perform the following on valid DNA Sequences: - Accept 'n' DNA sequences from the user. - Verify whether each sequence contains only the valid DNA bases (A, C, G, T). If so, display as Valid otherwise invalid. - Search for the specific "TATAAA" sequence within the given DNA sequences. If so Display as match/nomatch - Count and display the occurrences of base 'A' in each DNA sequence. - Store the unique bases and their frequency in a dictionary for all sequences combined and display them.

	• Replace a specific substring in the DNA sequences with 'X' (e.g. replacing "AT" in "TATA" results in "TXA"). • Convert DNA to RNA by replacing 'T' with 'U', and store the RNA sequences in a list. • Search for a specific pattern in the DNA sequences and display its position if found. • Compute and display the complementary strand for each DNA sequence, where A pairs with T, and C pairs with G (e.g., "ATCGA" → "TAGCT").
11.	Renewable Energy Grid (M3_CO2) In the year 2050, the world is powered by solar energy storage units, distributed across three major smart power stations. These stations are: Station 1: Primary source Station 2: Intermediate (buffer) Station 3: Final destination To ensure sustainable and stable energy flow, scientists have developed a precise transfer protocol that moves energy units from Station 1 to Station 3. Transfer Rules (Constraints): • Only one energy unit can be transferred at a time to prevent system overload. • A larger unit can never be placed on top of a smaller unit. • The number of energy units n must be an integer between 1 and 20. • The transfer output must be a sequence of moves: each move is a pair (a, b) meaning "move a unit from Station a to Station b", where a and b are in {1, 2, 3}. • Two consecutive moves must not involve the same pair of stations in opposite directions. For example, (1, 3), (3, 1) back-to-back is not allowed, as it causes unnecessary energy loss and instability.
Data Handling Strategies Using Lists, Tuples, Dictionary and Set- Data Comprehension - Iterators - Data Selection - Ordered, Unordered and Unique Data Organization - Data Modification - Grouping and Categorization – Searching and Sorting Techniques (14 hours)	
12.	Rainfall prediction using Sorting Write a Python program to assist meteorologists in forecasting rainfall by analyzing relative humidity based on air temperature (23°C to 28°C) and dew point temperature (7°C to 16°C). Use the given formulas to compute specific humidity, saturation point, and relative humidity: specific humidity = $6.11 \times 10^{\{7.5 \times \text{dew point}\} / \{237.3 + \text{dew point}\}}$ saturation point = $6.11 \times 10^{\{7.5 \times \text{air temp}\} / \{237.3 + \text{air temp}\}}$ relative humidity = (specific humidity) / (saturation point) x 100 The chance of rainfall is high when the relative humidity is high and the temperature is dropping. The program should sort the relative humidity values in increasing order and display the highest relative humidity along with its corresponding air temperature and dew point temperature, providing valuable insights for rainfall prediction.
13.	Browser Tracking

	Write a Python program to simulate a web browser's back and forward navigation system using stacks, where visiting a new webpage pushes the URL onto the back stack and clears the forward stack. Implement functions to navigate back by popping the top page from the back stack and pushing it onto the forward stack, and to navigate forward by popping the top page from the forward stack and pushing it onto the back stack. The program should allow users to enter URLs, navigate back and forward, and display the current page along with available navigation options dynamically. • An integer N representing the number of webpage visits. • N lines, each containing a URL (string) representing a webpage visit. • User can enter the following commands dynamically after initial webpage visits: ◦ BACK: Navigate to the previous page. ◦ FORWARD: Navigate to the next page. ◦ VISIT <URL>: Visit a new webpage, clearing the forward history. ◦ EXIT: Terminate the program.
14.	Customer Orders Queue Processing In a smart manufacturing unit, automated machines process customer orders in a First-In-First-Out (FIFO) manner to ensure efficiency. Write a Python program using a queue to simulate an order processing system, where customers place orders for different industrial products (e.g., raw materials, electronic components, machinery parts), and each order is stored along with its order placed time. The system should process the orders in the same sequence they were received, record the order completion time, and remove them from the queue upon completion. The system must accept an integer '<i>N</i>' representing the number of orders, <i>N lines, each containing</i>: • Order ID (string) • Product Name (string) • Order Placed Time (HH:MM 24hr -format) • Order Completion Time ((HH:MM format) Finally, display the status of the order queue before and after processing, along with the time taken to complete each order.
15.	Smart Building Elena Vasquez, a futuristic architect, is designing modular smart buildings on a blueprint. Each building is represented as a rectangle with: • (x, y) — coordinates of the bottom-left corner • width and height — horizontal and vertical sizes of the building To avoid overlap during construction, Elena needs a tool that calculates the exact overlapping area between any two buildings. Write a Python function (<i>collections framework</i>) that takes two building rectangles as input and returns the area of overlap. If they do not overlap, return 0.
16.	Analyze Water Quality Environmental agencies, researchers assess the overall water quality, monitor potential pollution sources, and ensure the water is safe for intended uses like drinking, agriculture, and aquatic life using water pH and dissolved oxygen levels and turbidity. Assist them by developing a Python program using <i>functions</i> and conditional statements to analyse the water quality

	based on pH, turbidity, and dissolved oxygen levels. The program should take input values for these parameters, check if they are within safe limits, and classify the water as safe or unsafe for consumption. If pH value is 6.5 and 8.5, Turbidity lesser than 3 NTU (Nephelometric Turbidity Unit) and Dissolved Oxygen between 6.5 and 8 mg/L then the water quality is safe Use conditional statements to give recommendations if the water is safe or unsafe. Allow users to input data for any location and display a summary of safe and unsafe water sources. Locations may include Clean River area, lake near agriculture area, urban storm drain area, Deep Ground water Source etc., Use menu driven approach and functions to compute and display for multiple locations.
Fundamentals of Function: Definition, Call, Return Value – Parameters and Arguments: Positional arguments, Keyword arguments, Parameters with Default Values, Arbitrary Arguments – Local and Global Scope - Lambda Functions – Decorators - Recursive Functions. (14 hours)	
17.	Reforestation In a reforestation project, using a tree we need to cultivate a fixed number of saplings every year. These saplings become mature trees starting from the next year and also begin producing new saplings. Write a Python program using functions that calculates the total number of mature trees after a given number of years by accepting a tuple of three integers: initial_trees – Number of trees at year 0 saplings_per_tree – Number of saplings each tree produces per year years – Number of years to simulate
18.	Recurring Character For a given string which is made of uppercase letters (A–Z). Write a recursive function that finds the first character that appears more than once. The function should return a dictionary with: • The character (key) • A list of two indices: • The index where it first appeared • The index where it reappeared first • If no recurring character is found, or if the input is None or an empty string, return an empty dictionary {}. Rules & Constraints: • Input must be a string containing only uppercase A–Z. • Return the first recurring character based on position. • Ignore later repetitions after the first recurrence. • Use recursion (no loops allowed). • Return an empty dictionary if: • Input is None • Input is an empty string • No character repeats
19.	Tower of Hanoi In robotic warehouse automation, autonomous robotic arms move stacked containers or pallets between storage locations while following strict stacking rules. Similar to the Tower of Hanoi problem, the system must transfer stacked items from a source rack to a destination rack using an intermediate rack, ensuring that larger items are never placed on smaller ones. Write a

	Python program using a recursive function to simulate this process, where a set number of stacked containers (disks) must be moved while following the <i>First-In-Last-Out (FILO) principle</i>. The program should determine the optimal sequence of moves, ensuring safe and efficient stacking, and display each step of the movement process to track how robotic arms relocate items.
Numpy: Array Operations – Mathematical Functions - Pandas: Handling Files - Creating Dataframes – Data Cleaning – Filtering – Selection – Grouping – Sorting – Aggregation – Merging (10 hours)	
20.	Student Attendance Data Analysis Involve as a team to collect attendance dataset from professors. Design a Python program using <i>NumPy to analyze</i> a student attendance dataset. • An integer N representing the number of students. • An integer D representing the number of days. • A N x D matrix, where each row corresponds to a student, and each column represents a day. ◦ 1 indicates present ◦ 0 indicates absent Compute the total attendance per student, identify the student with the highest attendance, calculate the class average attendance per day, and determine the day with the lowest attendance rate. Finally, compute each student's attendance percentage and display students who have less than 75% attendance.
21.	Student Marks Data Analysis using Pandas Involve as a team to collect sample archived student marks dataset from professors. Write a Python program using Pandas to analyse students' internal marks, where each student is evaluated based on six internal components and a final component. Create a DataFrame with student names, six internal scores, and final score. Calculate the class average internal score, compute the total marks for each student by summing all components, and determine the student with the highest total marks. Additionally, filter and display students who have scored below 40% indicating the need for improvement.
22.	Restaurant Food Waste Cost Restaurants throw away a lot of food every day, which affects both costs and the environment. Consider the dataset which contains details of food waste over a month. Write a <i>Python program using Pandas</i> to analyze and reduce this waste. Input Format: A CSV file with the following columns: • Date (YYYY-MM-DD) – The day waste was recorded • Food Item – Name of the food item wasted • Category – Type of food (e.g., Vegetables, Dairy, Meat) • Quantity Wasted – Amount wasted (in kg or units) • Cost per Item – Cost of one unit of the food item Perform the following: 1. Clean the Data ◦ Handle missing values in Date, Food Item, Category, Quantity Wasted, or Cost per Item (e.g., fill or remove). 2. Calculate Waste Cost ◦ Per Day: Total waste cost for each day ◦ Per Category: Waste cost for each food category

	3. Top 5 Most Wasted Items - Find the 5 food items with the highest total quantity wasted - Suggest simple strategies to reduce waste for each (e.g., store better, reuse, order less) 4. Trend & Strategy Analysis - Spot patterns (e.g., which days waste the most) - Recommend ways to reduce waste (e.g., smaller portions, better planning, donation)
23.	Smart Energy Grid (M5_CO3) A smart energy grid tracks hourly electricity usage (in kWh) for a residential building over 30 days. The data is stored in a NumPy array. Write a Python program that analyzes the energy usage using NumPy. Input Format: A NumPy array with shape (30, 24): 30 rows = 30 days 24 columns = 24 hours per day Each value shows the electricity used during that hour. Perform the following: Calculate Energy Usage Daily Consumption: Total energy used per day (output: array of 30 values) Monthly Consumption: Total energy used in 30 days (output: one float) Find Peak Hours For each day, find hours where usage is above that day's average Output: A dictionary {day_index: [list_of_peak_hours]} Predict Next Day's Usage Use the last 7 days to predict the next day's total energy usage using simple average of daily totals from the last 7 days
	Team Projects (Indicative) Creation of a management portal for admin/client stakeholders with Command Line Interface (CLI) based interactions can be suggested for the project implementation. Usage of dataset (publicly available or collected through survey process can be encouraged for team participation) and python language libraries can be explored as outcome-based learning instigation during implementation of the project. Menu options can be considered by the team during the implementation, where these menu options can represent either client/admin activities in the portal management page. Individual contribution of the team members (for each portal page interaction for the management of the portal) can be mentioned during the report submission. Any topic from the below mentioned list (but not limited to) can be considered. - Healthcare/ Vaccination portal admin/client access management - Census portal admin/client access management - Transportation portal admin/client access management - Election Board/Polling portal admin/client access management - Electricity/Water/Gas Consumption portal admin/client access management

	6. University/Examination portal admin/client access management 7. Meteorological Department portal admin/client access management 8. Exports/Imports/Trade portal admin/client access management		
Total hours:			60 hours
Mode of Evaluation: QUIZ/TEAM PROJECT/PAT/CAT/FAT			
Recommended by Board of Studies			
Approved by Academic Council	No.	Date	

Benchmark Details

SNo	Benchmarking Institute	Approximate % of Matching	URL for the Syllabus
1	NUS	80%	https://ace.nus.edu.sg/course/python-programming/
2	University of Oxford Carnegie, Purdue	80%	https://www.conted.ox.ac.uk/courses/python-programming-for-data-science-introduction
3	Stanford University	55%	https://uit.stanford.edu/service/techtraining/class/python-beginners
4	IIT Kanpur	60%	https://eicta.iitk.ac.in/product/programming-in-python/
5	IIT Madras	60%	https://study.iitm.ac.in/ds/course_pages/BSCS1002.html
6	Harvard University	55%	https://pll.harvard.edu/course/cs50s-introduction-programming-python

COURSE ARTICULATION MATRIX

Course Outcomes	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PO 12
CO1	3	3	3	2	3	0	0	0	3	2	0	3
CO2	3	3	3	2	3	0	0	0	3	2	0	3
CO3	3	3	3	2	3	0	0	0	3	2	0	3

Signature